

確信度ベース検証順序最適化による 効率的コード生成候補選択

概要: 大規模言語モデル (LLM) によるコード生成では、複数候補を生成し、コンパイルとテストで採用候補を選ぶ運用が一般的である。本研究では、LLM が返す token-level log 確率を用いた候補順位付けを評価し、log 確率がコードの意味的正しさをどこまで予測できるかを調べる。候補全体の平均 log 確率で検証順序を決める Mean Log-Probability Ranking (Mean)、局所的な低確率 token を考慮する Tail Log-Probability Ranking (Tail)、候補中で最も低い token log 確率を用いる Minimum Log-Probability Ranking (MinLogP) を比較する。主評価では、行列・疎行列・ソート・グラフ・数値計算を含む C 言語カーネル 20 問を用い、その中に ParEval 由来の問題と本研究で作成した数値・検証問題を含めた。各問題について、同一プロンプトを独立に呼び出して得た $k = 10$ 個の候補集合上でオフライン評価した。20 問を 5 問ずつ 4 ケースに分類すると、LogP-Separable では Tail が平均テスト数を Gen の 3.51 回から 2.65 回へ削減し、Tail-Sensitive でも Gen の 6.42 回を 5.67 回へ削減した。Weak-Signal では MinLogP が Gen の 2.86 回を 2.45 回へ削減し、Unreliable-Signal では MinLogP が Gen の 6.56 回を 6.50 回へわずかに削減した一方、Mean と Tail は悪化した。これらの結果は、log 確率は検証順序の軽量な手がかりにはなるが、意味的誤りの排除には不十分であり、効果は問題依存であることを示す。

1. はじめに

LLM のコード生成能力は、Codex[1] や AlphaCode[2] 以降、大きく向上している。実用的なコード生成では、単一の出力をそのまま採用するのではなく、複数の候補を生成し、コンパイル、ユニットテスト、性能テストを通して採用候補を選択することが多い。この候補生成とテストに基づく選択は品質を高める一方で、候補数に比例して実行コストが増加する。

特に C 言語性能最適化では、ループ展開、分岐削減、SIMD 命令、キャッシュブロッキング、アルゴリズム変更など、多様な実装方針が存在する。LLM はこれらの候補を複数生成できるが、各候補の品質は一様ではない。生成順にテストするだけでは、成功候補が後方にある場合に余分なテストが必要となる。一方で、全候補をテストして最良候補を選択する方法は、成功率の観点では堅牢だが、テストコストが高い。

本研究の着眼点は、LLM が各トークンに対して出力する log 確率である。高 log 確率のトークン列は構文的に自然である可能性が高いが、意味的正しさを保証しない。そこで本研究では、平均 log 確率、低確率 token の裾、最小 token log 確率に基づく 3 種類の順位付けを比較し、token log 確率が候補の正しさをどの程度識別できるかを評価する。

本研究の貢献は以下の 3 点である。

- **log 確率ベース順位付けの限界分析:** Mean, Tail, MinLogP を比較し、高 log 確率でも意味的に誤る候補が問題依存で残ることを示す。
- **効果が出る問題特性の分析:** 20 問を LogP-Separable, Tail-Sensitive, Weak-Signal, Unreliable-Signal の 4 ケースに分類し、log 確率順位付けが有効になる条件を分析する。
- **同一候補集合に基づく評価:** 候補生成の乱数差を排除するため、一度生成した候補集合を共有し、検証順序だけがテスト回数に与える影響をオフライン比較する評価手順を構築した。

以下、第 2 章で関連研究、第 3 章で予備知識、第 4 章で提案手法、第 5 章で実験設定、第 6 章で評価結果、第 7 章で考察、第 8 章で結論を述べる。

2. 関連研究

2.1 不確実性を用いた LLM 出力制御

Semantic Entropy[3] は、意味的に同じ出力をクラスタリングし、意味空間でのエントロピーを計算することで LLM の幻覚検出精度を向上させた。Code Semantic Entropy[4] は、この考え方をコード生成に応用し、プログラムの振る舞いの多様性に基づいて問題レベルの不確実性を測定する。Token-level 不確実性に関しては、TokUR[5] や token-level uncertainty-aware objective[6] が、低ランクランダム重み

摂動や aleatoric/epistemic 不確実性の分解により、トークン単位の信頼度評価や訓練目的への応用を検討している。

コード生成への不確実性の応用として、USCD (Uncertainty-Aware Selective Contrastive Decoding) [7] は、token-level 不確実性を用いて出力ノイズを除去し、negative prompt との対比的デコーディングを実施する手法を提案した。また、Sharma ら [8] は、自然言語生成の不確実性推定技術をコード生成に適応し、symbolic execution で意味的等価性をクラスタリングすることで、LLM 出力の正しさを不確実性で評価する手法を提案した。

本研究は、これらの token-level 不確実性計測手法を生成後候補の検証順序最適化に適用する点で関連する。

2.2 適応的デコーディング戦略

AdaDec[9] は、token-level 不確実性 (Shannon entropy) を用いたコード生成時の適応的デコーディング手法である。高不確実性を検出したときに pause-then-rerank 機構を実行し、lookahead によりトークン選択を改善する。AdaDec はデコーディング中のリアルタイム介入により単一候補の品質を向上させる手法である。

AdapT[10] や EDT[11] は、トークンごとまたはステップごとに温度を適応的に調整する手法であり、生成品質と多様性のバランスを取る。Semantic uncertainty in advanced decoding methods[12] は、CoT 等の高度なデコーディング手法における semantic uncertainty を調査し、デコーディング戦略が出力の多様性と信頼性に与える影響を分析した。これらの手法は生成過程の制御を目的とする点で、本研究の生成後順位付けとは異なる。

2.3 候補選択と反復的改善

Self-Consistency[13] は、複数の推論経路をサンプリングし、最も一貫性のある解答を選択する手法で、コード生成でも複数候補選択の有効性を示唆する。Codex[1] や AlphaCode[2] は複数候補を生成し、テスト実行やクラスタリングで絞り込むが、候補の優先順位付けに LLM の確率情報を直接使用していない。CodeT[14] は生成テストを用いて候補を評価し、CodeRL[15] はテスト結果を報酬として利用する。本研究は、テスト前に得られる token-level 確率だけで生成済み候補を順位付けする点で異なる。

反復的改善に関しては、Self-Refine[16] が LLM 自身による反復的フィードバック・改善を提案したが、コード全体を対象とし、検証順序の最適化は扱わない。UnCert-CoT[17] は不確実性定量化を取り入れ、挑戦的な側面を特定して選択的に CoT 推論を活性化する。これらの反復的改善に対し、本研究は生成済み候補の検証順序のみを変更する後処理として token-level 不確実性を利用する。

2.4 確率較正と LLM の信頼性

LLM の確率較正 (calibration) は、出力確率が実際の正解率と一致するかを評価する。Kadavath ら [18] は、言語モデルが自身の知識の境界を一定程度認識できることを示し、Xiong ら [19] は LLM の不確実性表現能力を包括的に評価した。SPUQ[20] は入力摂動により不確実性を推定し、Expected Calibration Error (ECE) を平均 50%削減した。

コード生成タスクでの確率較正の検証はまだ限定的である。本研究は、高確信候補が実際に高品質であるかを、候補検証順序と成功率・テスト回数の観点から評価する。既存研究と比較した本研究の特徴は、生成後候補集合に対する軽量のテスト順序最適化を扱う点である。

3. 予備知識

3.1 大規模言語モデルとコード生成

大規模言語モデル (LLM) は、Transformer アーキテクチャを基盤とし、大規模なテキストコーパスとソースコードで事前学習された自己回帰型の生成モデルである。コード生成では、自然言語の問題記述、関数シグネチャ、制約条件、入出力例などを入力として、対応するプログラムをトークン列として生成する。

自己回帰生成では、モデルはそれまでに生成したトークン列 $t_1, \dots, t_{i-1}$ に基づき、次トークン t_i の条件付き確率 $P(t_i | t_1, \dots, t_{i-1})$ を計算する。生成時には、貪欲法、beam search、サンプリングなどにより次トークンが選択される。本研究では複数候補を得るため、温度付きサンプリングを用いる。

温度パラメータ T を用いたサンプリングでは、logits s_i から以下の分布を計算する。

$$P(t_i | t_1, \dots, t_{i-1}) = \frac{\exp(s_i/T)}{\sum_j \exp(s_j/T)} \quad (1)$$

T が大きいほど分布は平坦になり、多様な候補が生成されやすい。一方、 T が小さいほど高確率トークンに集中し、生成は決定的になる。本研究の主評価では、候補の多様性と品質のバランスを取るため、temperature=0.8 を用いる。

3.2 logprobs と確信度計算

多くの LLM 推論エンジンは、生成各トークンの log 確率 (logprobs) を返す機能を提供する。logprobs は、そのトークンが選択される対数確率 $\log P(t_i | t_1, \dots, t_{i-1})$ であり、モデルが各トークンをどの程度の確信を持って生成したかを表す。

本研究では、候補コード c の確信度を以下で定義する。

$$\text{conf}(c) = \exp\left(\frac{1}{N} \sum_{i=1}^N \log P(t_i)\right) \quad (2)$$

ここで N は生成されたトークン数である。この値は、全

トークン確率の幾何平均に相当する。単純な確率積を用いると長い候補ほど不利になるため、平均 log 確率を先に計算し、最後に指数化する。実装では、logprob が取得できない特殊トークンや None 値を除外して計算する。

高確信度のコードは、学習データ中で頻出する構文や識別子の使い方と整合している可能性が高い。また、モデルが実装方針や境界条件で迷う場合、局所的な token 確率が低下することがある。一方で、構文的に自然なコードでもアルゴリズムや境界条件が誤ることはあるため、log 確率は候補品質の十分条件ではない。本研究ではこの限界を含めて実験的に評価する。

4. 提案手法

本稿では、LLM が出力する token-level log 確率から候補コードのスコアを計算し、そのスコアに基づいて検証順序を制御する。扱う選択規則は、平均 log 確率を用いる Mean Log-Probability Ranking (以下, Mean), 低確率 token の裾を用いる Tail Log-Probability Ranking (以下, Tail), 候補中で最も低い token log 確率を用いる Minimum Log-Probability Ranking (以下, MinLogP) である。いずれの規則も生成済みの固定候補集合を並べ替えるだけで、候補生成自体は変更しない。

4.1 Mean Log-Probability Ranking

Mean は、候補 c の平均 log 確率に基づく確信度順に検証する手法である。候補プール内の成功候補が高確信度側に偏っていれば、生成順より少ないテスト数で成功候補に到達できる。

Algorithm 1 に Mean を示す。まず LLM から k 個の候補を生成し、各候補の logprobs から平均 log 確率に基づく確信度を計算する。次に候補を確信度の降順に並べ替え、順にコンパイル・テストする。成功候補が見つければ、その時点で終了し、残りの候補の検証を省略する。全候補が失敗した場合は失敗として扱う。

以降の Tail と MinLogP も、Algorithm 1 の整列基準だけを置き換えた規則として実装できる。

4.2 Tail Log-Probability Ranking

平均 log 確率は候補全体の自然さを表す一方で、局所的に極端に低確率な token の存在を十分に反映しない場合がある。そこで Tail では、候補 c の token log 確率列 $\{\ell_i\}_{i=1}^N$ に対し、下位四分位

$$\text{tail}(c) = Q_{0.25}(\{\ell_i\}_{i=1}^N) \quad (3)$$

を局所 tail 確信度として用いる。Tail は $\text{tail}(c)$ の降順に候補を検証し、平均的には自然でも一部に低確率 token を含む候補を後回しにする。この規則は、候補の大半が自然な token 列で構成されていても、局所的な低確信度 token が

Algorithm 1 Mean Log-Probability Ranking

Require: 問題記述 P , 候補数 k

Ensure: 成功したコード候補または失敗

```
1: 候補集合  $C = \{c_1, \dots, c_k\}$  を LLM で生成し, logprobs を取得
2: for 各候補  $c_j \in C$  do
3:    $\text{conf}(c_j) = \exp(\frac{1}{N_j} \sum_i \log P(t_i))$  を計算
4: end for
5:  $C$  を  $\text{conf}(c_j)$  の降順に整列
6: for 各候補  $c_j \in C$  do
7:    $c_j$  をコンパイルし, テストを実行
8:   if  $c_j$  が成功 then
9:     return  $c_j$ 
10:  end if
11: end for
12: return 失敗
```

バグや境界条件の誤りに対応する場合に有効であることを期待する。

4.3 Minimum Log-Probability Ranking

Tail は下位四分位を用いるため、候補全体に広がる低確信度 token の傾向を反映する。一方で、コンパイル失敗や局所的な実装破綻は、少数の極端に低い token log 確率として現れる場合がある。そこで MinLogP では、候補 c の token log 確率列 $\{\ell_i\}_{i=1}^N$ に対し、

$$\text{minlogp}(c) = \min_{1 \leq i \leq N} \ell_i \quad (4)$$

をスコアとして用いる。MinLogP は $\text{minlogp}(c)$ の降順に候補を検証する。これは、候補中の最も低確信度な token が相対的に高い候補を優先する規則であり、最大 token loss $\max_i -\ell_i$ が小さい候補を優先することと等価である。しきい値や罰則係数を導入しないため、極端低確信度 token を扱う手法として単純で、問題ごとのパラメータ調整を必要としない。

5. 実験設定

5.1 環境

実験では、Qwen3.5-9B-AWQ-4bit を vLLM[21] 経由で利用した。vLLM は PagedAttention を用いた高速な LLM 推論エンジンであり、OpenAI 互換 API として利用できる。本実験では chat completion API を使い、logprobs パラメータにより生成トークンの log 確率を取得した。各候補について生成テキストと token-level logprobs を同時に保存し、生成後にコードブロック抽出、確信度計算、コンパイル・テストを行った。

生成パラメータは temperature=0.8、候補数は全評価で $k = 10$ に統一した。thinking 出力は chat template の設定で無効化し、max.tokens は 4096 とした。生成された C コードは gcc 14.3.1 で -O3 -lm を付けてコンパイルし、各問題に付属するテストプログラムを実行した。実験環境は Linux 6.12, Python 3.12.12, Intel Xeon Platinum 8368

CPU である。主評価では 20 問それぞれについて 100 試行を行い、合計 2000 試行を評価した。コード生成時の乱数 seed は vLLM API 側で固定していないため、完全な再生成再現性は保証しない。一方、主評価では全候補のコード、確信度、コンパイル・テスト結果、エラー種別を JSON に保存し、解析の再現性を確保した。

5.2 比較手法

ここで、主評価における候補の生成方法を明示する。各問題には、後述の通常の最適化指示プロンプト P がある。主評価では、プロンプト内で複数解の列挙を要求するのではなく、各問題・各試行について同じプロンプト P を $k = 10$ 回独立に API へ送信し、1 回の呼び出しから 1 個の候補を得る。これにより候補集合 $C = \{c_0, \dots, c_{k-1}\}$ を構成する。

本稿の比較手法はすべて、この同一候補プールに対する**選択規則**の違いとして定義する。実験ではまず全候補をコンパイル・テストして成否と log 確率統計を保存し、その後に各規則をオフライン評価する。

- **Gen**: 同一プロンプトから得た共有候補プールを API の返却順 $c_0, \dots, c_{k-1}$ にテストし、最初に成功した候補を採用する生成順早期停止である。
- **Mean**: 共有候補プール全体を使い、平均 log 確率に基づく確信度の降順に候補をテストする Mean Log-Probability Ranking である。
- **Tail**: 共有候補プール全体を使い、token log 確率の下位四分位 $\text{tail}(c)$ の降順に候補をテストする Tail Log-Probability Ranking である。
- **MinLogP**: 共有候補プール全体を使い、候補中で最も低い token log 確率 $\text{minlogp}(c)$ の降順に候補をテストする Minimum Log-Probability Ranking である。

この設計により、候補集合の違いによる成功率の揺らぎを排除し、検証順序だけがテスト数に与える影響を評価できる。

5.3 評価タスク

表 1 に、本研究で用いる 20 問の評価タスクを示す。評価対象は C 言語カーネルの実装タスクであり、指定された関数シグネチャに従って関数または関数群を実装し、テストプログラムとリンクしてコンパイル・実行する。問題には、疎行列ベクトル積、キャッシュを意識した転置、ソート、グラフ処理、ステンスル計算、数値線形代数、文字列検証などが含まれる。この 20 問には、ParEval から取り込んだ問題、本研究で新たに作成した数値・検証問題、および一般的な最適化カーネルを題材にした問題を含める。

本稿では、コードの題材だけでなく、log 確率に基づく順位付けがどのように効いたかを分析するため、20 問を 5 問ずつ 4 ケースに分類する。これは事前難易度ではなく、

候補集合上で観察された確信度信号の振る舞いに基づく分析用分類である。

- **LogP-Separable**: Mean または Tail だけで Gen より明確に少ないテスト数を達成した問題群である。成功候補と失敗候補の log 確率統計が比較的分離する。
- **Tail-Sensitive**: 成功候補が少ない、またはテスト数が高めに残るが、低確信度 token の裾を用いる Tail が比較的安定して改善する問題群である。
- **Weak-Signal**: log 確率順位付けで小幅な改善は得られるが、Gen との差が小さく効果が不安定な問題群である。
- **Unreliable-Signal**: log 確率に基づく順位付けがほぼ効かない、または Gen より悪化する問題群である。意味的誤りが自然な token 列として生成される場合や、候補集合内の成功候補が極端に少ない場合を含む。

この分類により、単に成功率の高低で分けるのではなく、Mean, Tail, MinLogP がどのような候補集合で有効になるかを分析できる。

5.4 プロンプト設計

生成プロンプトは、各問題の目的、関数シグネチャ、実装上の指示、出力制約からなる。典型的には、“Implement an optimized ... in C” で最適化対象を指定し、必要な関数シグネチャと正しさに関わる条件を列挙する。最後に、C コードのみを出力し、必要な `#include` と補助定義は許可し、Markdown フェンスや説明文を含めないという最小限の制約を付ける。

例えば、Strided Transpose では、入力要素 $\text{in}[\text{i} \cdot \text{in_stride} + \text{j}]$ を出力要素 $\text{out}[\text{j} \cdot \text{out_stride} + \text{i}]$ へ書き込み、padding 領域を変更しないよう指示する。Radix では、キーと値の対応を保持した安定ソートを実装するよう指示する。Sort Nonzero では、ゼロ値の位置を保持しながら非ゼロ要素のみを整列する制約を明示する。RMSNORM では、float 入出力に対して double 蓄積を用い、テストで要求される誤差閾値を満たすよう指示する。UTF-8 検証では、過長符号化や surrogate を拒否する境界条件を明示し、Cholesky 分解では非正定値入力を検出して失敗を返すよう指示する。Convex Hull では重複点や共線点の扱いを明示する。このように、プロンプトは単なる関数名だけでなく、期待する最適化方針と正しさに必要な条件を含む。

5.5 評価指標

成功率は、コンパイルに成功し、テストプログラムが PASS を出力した候補を採用できた試行の割合である。本稿の比較手法は同一候補集合の順序だけを変えるため、成功率は手法別の性能差ではなく、候補集合自体の品質を表す補助指標である。平均テスト数は、各選択規則が成功ま

表 1 生成対象となる C コードの内容 (20 課題)

ベンチマーク	集合	カテゴリ	生成対象コード
Sort Nonzero	LogP-Separable	Sorting	ゼロ位置を固定したまま、非ゼロ要素のみを昇順に並べ替える。
UTF-8 Validate	LogP-Separable	Validation	UTF-8 列について過長符号化, surrogate, 範囲外 code point を厳密に拒否する。
Largest Component	LogP-Separable	Graph	グリッドまたはグラフ上の連結成分を探索し、最大成分サイズを返す。
SpMV	LogP-Separable	Sparse	CSR 形式の疎行列ベクトル積, AXPY, 内積を一回の走査で融合する。
Strided Transpose	LogP-Separable	Stride	入出力 stride を持つ矩形行列転置を実装し、出力 padding を保持する。
Convex Hull	Tail-Sensitive	Geometry	重複点や共線点を含む点集合に対して凸包周長を計算する。
Top-k NaN	Tail-Sensitive	Selection	NaN を無視して浮動小数点配列の上位 k 要素を選択する。
Stencil3D	Tail-Sensitive	Stencil	3 次元 7 点ステンシルを境界条件と精度制約を満たして実装する。
Stencil2D	Tail-Sensitive	Stencil	halo 領域を含む 2 次元ステンシル計算を実装する。
Cholesky SPD	Tail-Sensitive	Linear Algebra	対称正定値行列の Cholesky 分解を行い、非正定値入力を検出する。
Crop2D	Weak-Signal	Stride	入力 stride と出力 padding を持つ 2 次元 crop を実装し、padding 領域を変更しない。
Transpose	Weak-Signal	Cache	行列転置をブロック単位に分割してキャッシュ局所性を高める。
RMSNorm	Weak-Signal	Reduction	RMSNorm の二乗和計算を double で蓄積して丸め誤差を抑える。
HeapSort	Weak-Signal	Heap	ヒープ構造の構築・維持とソート処理を正しく実装する。
Quadratic Roots	Weak-Signal	Numerics	桁落ちを避ける二次方程式の安定な解法を実装する。
Conv2D	Unreliable	Convolution	NHWC 配置の multi-channel 3x3 valid convolution を実装する。
Radix	Unreliable	Sorting	キーと値の対応を保持する安定 radix sort を実装する。
Floyd	Unreliable	Graph	全点対最短経路を Floyd-Warshall アルゴリズムで in-place 更新する。
Triangular Solve	Unreliable	Solver	複数右辺の下三角ソルバを stride 付き行列上で実装する。
Banded Edit	Unreliable	Dynamic Prog.	帯幅制約付き編集距離を計算し、帯外遷移を扱う。

たは全失敗を確定するまでに必要とする候補テスト数を、問題数と試行数で割った値である。全候補検証では常に k となり、早期停止方式では成功候補の順位に応じて小さくなる。

6. 評価

6.1 主要評価

各手法を同一候補プール上で評価する。図中では、生成順ベースラインを Gen, 平均 log 確率順位を Mean, 局所 tail 順位を Tail, 最小 token log 確率順位を MinLogP と略記する。4 ケースの評価結果を図 1-4 に示す。上段の確信度分布は候補コード単位のヒストグラムであり、横軸は各候補の token log 確率平均を指数化した確信度、縦軸は各 bin に入った候補数である。破線は confidence=0.75 を示し、タイトル下の括弧内にその閾値以上での成功率を示す。この閾値は高確信度領域の反例を可視化するための補助線であり、Mean, Tail, MinLogP の順位付けには使用しない。また、高確信度領域にある失敗候補を赤い rug マークで示す。下段は trial 単位のテスト数の箱ひげ図である。箱は四分位範囲、中央線は中央値、ヒゲは 1.5 IQR 内の範囲、色付きひし形は平均値を示す。

図 5 は、Mean と Tail が実際に参照する 2 つの log 確率統計量を候補単位で可視化した代表例であり、LogP-Separable, Tail-Sensitive, Weak-Signal, Unreliable-Signal から 1 問ずつ示す。Sort Nonzero では成功候補が比較的右上側に偏り、log 確率に基づく順位付けが有効に働く。Stencil3D では成功候補は少ないが、低確信度 token の裾を用いる Tail が成功候補を相対的に上位へ置きやすい。Crop2D では成功候補と失敗候補が重なり、log 確率信号は弱い。Floyd

では高確信度でも意味的に誤る候補が多く、log 確率だけでは検証前に失敗を十分に排除できない。図 6 は、同じ現象を token 単位で見た代表例である。失敗候補の曲線が成功候補より上側にある領域では、失敗候補が低確信度 token を多く含んでいることを意味する。Sort Nonzero や Stencil3D では失敗候補の裾が相対的に重く、Tail がこの差を利用できる。一方、Crop2D や Floyd では曲線が重なる、または成功候補側も同程度に低確信度 token を含むため、低確信度 token の裾だけでは成功候補を安定して上位に置けない。

図 1-4 より、log 確率に基づく順位付けの効果はケースによって大きく異なる。LogP-Separable では Gen の平均 3.51 回に対し、Mean は 2.70 回、Tail は 2.65 回、MinLogP は 2.79 回まで削減した。Tail-Sensitive では Tail が Gen の 6.42 回を 5.67 回へ削減し、候補集合内の成功候補が少ない問題でも一定の改善を示した。一方、Weak-Signal では MinLogP が Gen の 2.86 回を 2.45 回へ削減し、Mean や Tail より大きい改善を示した。Unreliable-Signal では Gen の 6.56 回に対し、Mean は 6.78 回、Tail は 6.74 回へ悪化した。MinLogP は 6.50 回で Gen をわずかに下回った。

6.2 Temperature と候補数の感度分析

Tail の効果が生成設定に依存するかを確認するため、Tail の効果が比較的大きな transpose_strided_f64 と utf8_validate_strict の 2 問を用いて、temperature と候補数 k を変えた感度分析を行った。各条件では各問題 20 試行ずつ、合計 40 試行を評価し、図 7 では 2 問をプールした平均テスト数を示す。

全体として、 k を大きくすると Tail が検証順序を改善

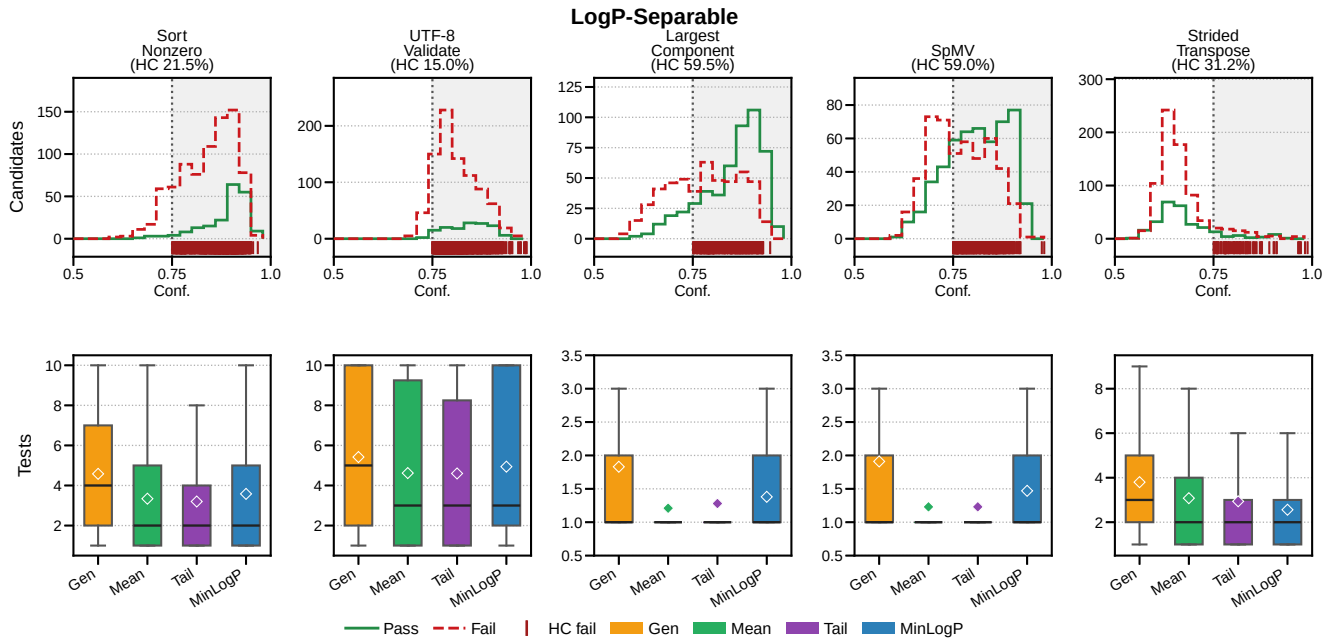


図 1 LogP-Separable ケース (Sort Nonzero, UTF-8 Validate, Largest Component, SpMV, Strided Transpose) の評価結果. 成功候補と失敗候補の log 確率統計が比較的分離し, Mean または Tail が Gen より大きくテスト数を削減した.

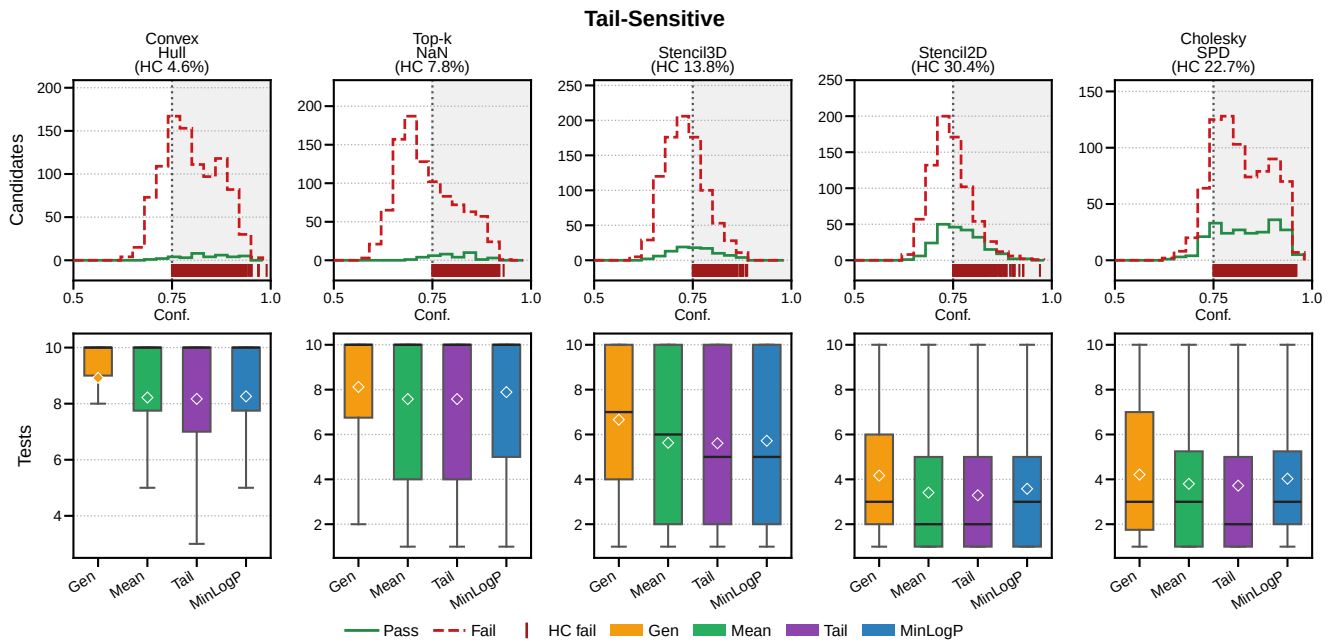


図 2 Tail-Sensitive ケース (Convex Hull, Top-k NaN, Stencil3D, Stencil2D, Cholesky SPD) の評価結果. 候補集合内の成功候補が少ない, またはテスト数が高めに残るが, Tail が Gen より安定して少ないテスト数を示す.

する余地が大きくなった. Gen では最初の 5 候補内に成功候補がある試行のテスト数は $k = 5$ と $k = 10$ で変わらないため, $k = 10$ での改善は Gen が大きく悪化したためではない. Tail は候補集合全体を順位付けするため, k が大きいほど選択可能な成功候補が増え, 成功候補を上位に配置できる機会が増える. 特に $k = 10$ では Tail gain が

0.52–0.55 となり, $k = 3$ や $k = 5$ より大きい改善が見られた. 一方, temperature=1.0 では $k = 3, 5$ で Tail gain が 0.07–0.10 に低下し, 高 temperature で候補品質のばらつきが大きくなると, log 確率だけによる順位付けが不安定になることが示唆される. 本実験では temperature=0.8 が最も安定して Tail の改善を示した. ただし, 本感度分析は

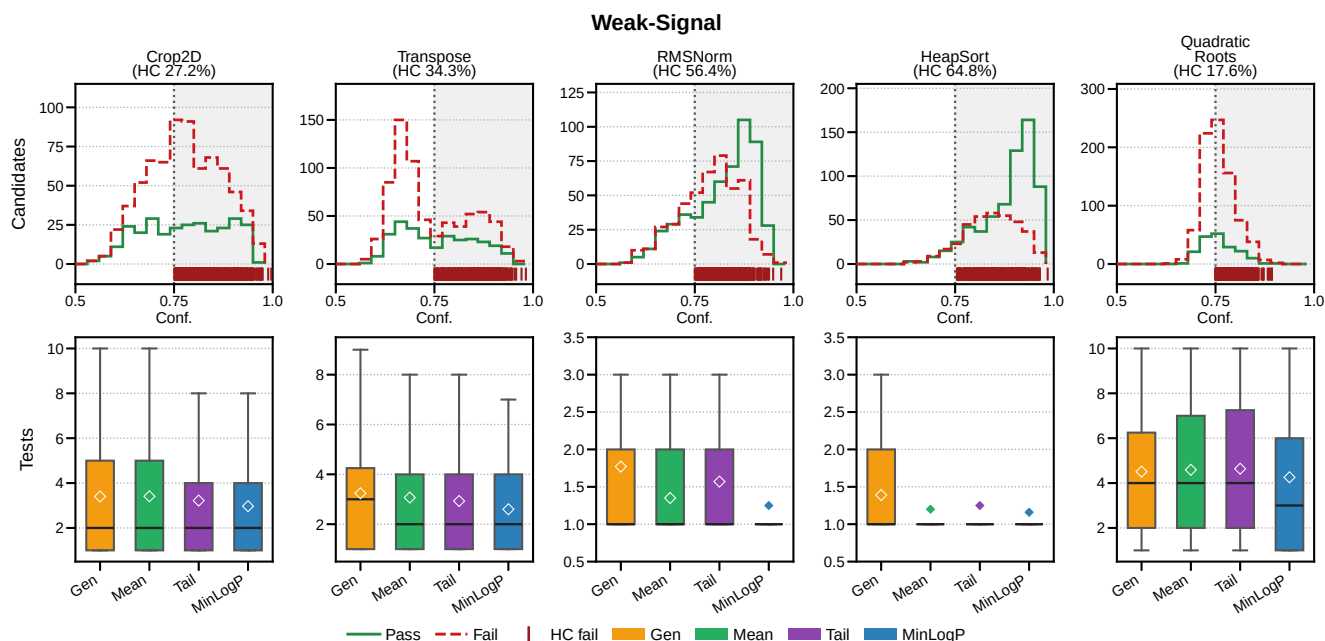


図 3 Weak-Signal ケース (Crop2D, Transpose, RMSNorm, HeapSort, Quadratic Roots) の評価結果. log 確率順位付けによる改善は小幅で, 最良手法も問題ごとに入れ替わる.

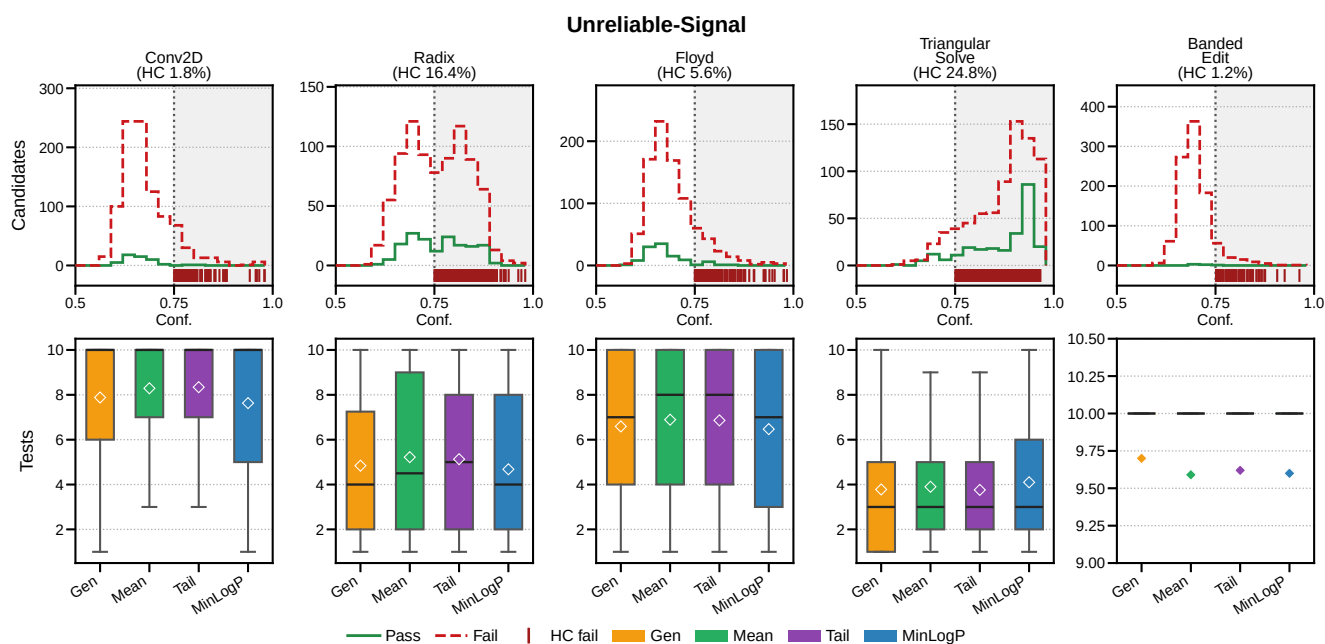


図 4 Unreliable-Signal ケース (Conv2D, Radix, Floyd, Triangular Solve, Banded Edit) の評価結果. log 確率に基づく順位付けがほぼ効かない, または Gen より悪化する問題群である.

2 問, 各条件 40 試行の予備実験であり, 主評価とは区別して解釈する必要がある.

7. 考察

100 試行の結果では, log 確率順位付けの有効性は平均的な効果量だけでなく, 問題ごとの信号の安定性を見る必要があることが明確になった. 本稿の 4 分類は, この候補集

合上で観測される log 確率信号の性質に基づくものである.

LogP-Separable では, 成功候補と失敗候補の MeanLogP または TailLogP が比較的分離し, Tail は平均テスト数を Gen の 3.51 回から 2.65 回へ削減した. この群では, モデルが高確信度で生成するコードの自然さが, 少なくとも候補間の相対順位として正しさに対応している. Tail-Sensitive では, 候補集合内の成功候補が少ないため絶対的なテスト

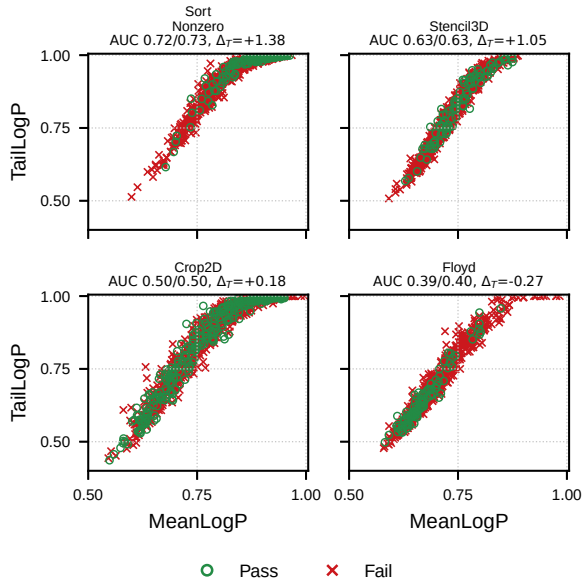


図 5 代表ベンチマークにおける各候補の MeanLogP と TailLogP の分布。9B モデル, $k = 10$ で評価した候補について、各点は 1 候補を表し、横軸は平均 token log 確率に基づく確信度、縦軸は token log 確率の下位四分位に基づく tail 確信度である。緑はテスト成功候補、赤は失敗候補を示す。各パネルの AUC は、成功候補を失敗候補より高く順位付けできる確率を Mean/Tail の順に示し、 Δ_T は Gen に対する Tail の平均テスト数削減量を表す。

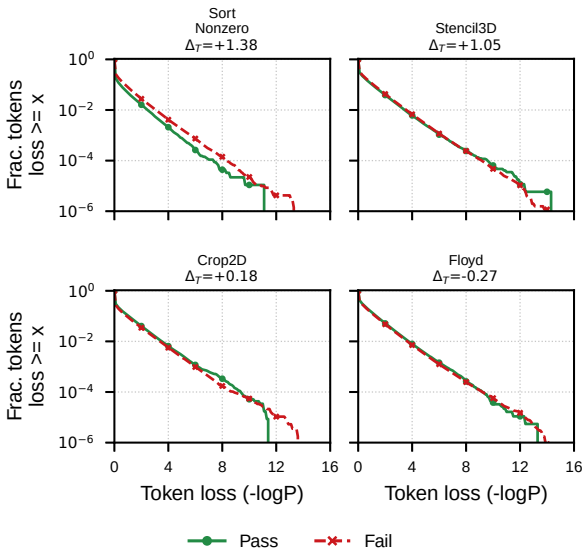


図 6 代表ベンチマークにおける全 token の低確信度側分布。成功候補に含まれる token と失敗候補に含まれる token を集約し、token loss $-\log P(t)$ の生存分布を示す。縦軸は loss が横軸値以上となる token の割合であり、右側の裾が長いほど低確信度 token を多く含む。全 token を点で描くと過密になるため、分布として集約した。

数は高めに残るが、低確信度 token の裾を使う Tail が Gen の 6.42 回を 5.67 回へ削減した。これは、候補全体の平均

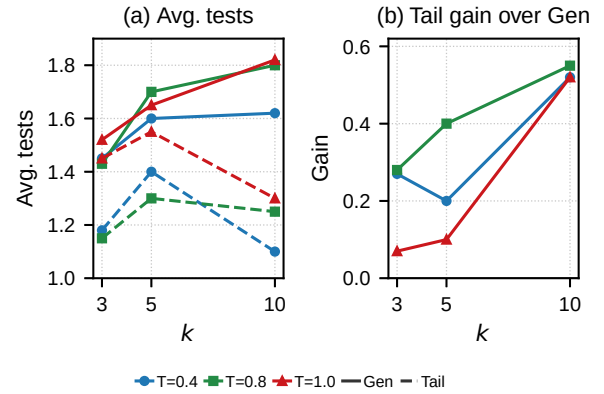


図 7 Temperature と候補数 k に対する感度分析 (2 問、各条件 40 試行)。左は 2 問をプールした Gen と Tail の平均テスト数、右は Gen に対する Tail の平均テスト数削減量を示す。

的な自然さよりも、局所的に怪しい token を含まないことが成功候補を早く見つける手掛かりになる場合を示している。

一方、Weak-Signal では MinLogP が Gen の 2.86 回を 2.45 回へ削減し、局所的に最も低い token log 確率が候補品質の手掛かりになる場合があることを示した。ただし、この群では問題ごとに最良手法が入れ替わるため、信号は補助的な手掛かりに過ぎない。Unreliable-Signal では、Mean と Tail は Gen より悪化し、MinLogP も 6.56 回から 6.50 回へのごく小さい改善にとどまった。この群では、Conv2D, Radix, Floyd のように構文的には自然で高 log 確率な実装でも、更新順序、安定性、境界条件、データ構造の保持といった意味的条件を満たさない候補が多い。また Banded Edit のように候補集合内の成功候補が極端に少ない場合、順位付けの余地自体が小さい。したがって、token log 確率は「モデルが迷った token」を検出するには有用だが、「自信を持って誤った実装」を検出するには不十分である。

実用上は、Mean, Tail, MinLogP はいずれも logprobs を取得し候補をソートするだけであるため、既存の vLLM ベースのコード生成パイプラインに容易に追加できる。ただし、今回の結果では Tail が LogP-Separable と Tail-Sensitive で有望である一方、MinLogP は Weak-Signal や Unreliable-Signal で Tail を補完した。したがって、log 確率順位付けは全問題に一律に適用する万能な選択器ではなく、問題ごとにどの token log 確率統計が成功候補と失敗候補を分けるかに依存する軽量ヒューリスティックとして位置付けるのが妥当である。本稿の評価は固定候補集合上のオフライン評価であり、生成候補数や壁時計時間の削減は対象外である。実運用での総実行時間を議論するには、生成時間、コンパイル時間、テスト時間を含めたオンライン評価が別途必要である。また、高確信度の意味的誤りをテスト前に検出するには、構文解析、型検査、静的解析、

生成テスト、仕様一致チェックを組み合わせた確信度拡張が必要である。

7.1 妥当性への脅威

本評価にはいくつかの制約がある。第一に、20問の4分類は実験結果の観察に基づく分析用分類であり、未知の問題を事前に分類できることを主張するものではない。本稿の主張は、log 確率信号の効き方が問題ごとに異なるという観察と、その代表的な失敗様式の整理に限定される。第二に、主評価は Qwen3.5-9B-AWQ-4bit, temperature=0.8, $k=10$ に固定しており、他モデル、量子化方式、デコーディング設定で同じ効果量が得られるとは限らない。感度分析はこの依存性を確認するための予備実験であり、汎化性を完全に示すものではない。第三に、本稿は同一候補集合上のオフライン評価であるため、成功率や生成回数の改善は主張しない。実運用での有効性は、logprobs 取得のオーバーヘッド、コンパイル時間、テスト時間の比率に依存する。第四に、token log 確率はモデル内部の相対的な信号として用いており、確率較正された正解確率とはみなしていない。特に、高 log 確率でも意味的に誤る候補が残ることは、本評価の重要な制約である。

8. 結論

本研究では、LLM の token-level log 確率を用いた候補順位付けとして、Mean, Tail, MinLogP を比較した。評価の結果、LogP-Separable では Tail が平均テスト数を Gen の 3.51 回から 2.65 回へ削減し、Tail-Sensitive でも Tail が Gen の 6.42 回を 5.67 回へ削減した。Weak-Signal では MinLogP が Gen の 2.86 回を 2.45 回へ削減した。一方、Unreliable-Signal では MinLogP の改善も小さく、高 log 確率でも意味的に誤る候補が残るため、log 確率だけで semantic correctness を十分に判別できない。

以上より、token log 確率は複数候補を生成してテストで選別するコード生成において有用な軽量信号であるが、意味的正しさの判定には限界がある。本稿の 20 問分類は、log 確率信号が有効なケースと誤誘導するケースの両方を含む評価であり、より強い一般化には追加問題、同一試行数での再評価、オンライン評価が必要である。今後は、log 確率に静的解析、生成テスト、仕様一致チェックを組み合わせ、高確信度だが意味的に誤る候補をテスト前に検出する方法へ拡張する必要がある。

参考文献

[1] Chen, M. et al.: Evaluating Large Language Models Trained on Code, Technical Report arXiv:2107.03374, OpenAI (2021).
[2] Li, Y. et al.: Competition-level code generation with AlphaCode, *Science*, Vol. 378, No. 6624, pp. 1092–1097 (online), DOI: 10.1126/science.abq1158 (2022).

[3] Farquhar, S. et al.: Detecting hallucinations in large language models using semantic entropy, *Nature*, Vol. 630, No. 8017, pp. 625–630 (online), DOI: 10.1038/s41586-024-07421-0 (2024).
[4] Zhang, H. et al.: Self-Improving Code Generation via Semantic Entropy and Behavioral Consensus, Technical Report arXiv:2603.29292, arXiv (2026).
[5] Zhang, T. et al.: TokUR: Token-Level Uncertainty Estimation for Large Language Model Reasoning, *Proc. 14th International Conference on Learning Representations (ICLR)*, (online), available from <https://openreview.net/forum?id=VHQc7wzmYv> (2026).
[6] Liu, T. et al.: Token-Level Uncertainty-Aware Objective for Language Model Post-Training, Technical Report arXiv:2503.16511, arXiv (2025).
[7] Wang, S. et al.: USCD: Improving Code Generation of LLMs by Uncertainty-Aware Selective Contrastive Decoding, Technical Report arXiv:2409.05923, arXiv (2024).
[8] Sharma, A. and David, C.: Assessing Correctness in LLM-Based Code Generation via Uncertainty Estimation, Technical Report arXiv:2502.11620, arXiv (2025).
[9] He, K. et al.: Towards Better Code Generation: Adaptive Decoding with Uncertainty Guidance, Technical Report arXiv:2506.08980, arXiv (2025).
[10] Zhu, Y. et al.: Hot or Cold? Adaptive Temperature Sampling for Code Generation with Large Language Models, *Proc. AAAI Conference on Artificial Intelligence (AAAI)*, (online), available from <https://arxiv.org/abs/2309.02772> (2024).
[11] Zhang, S. et al.: EDT: Improving Large Language Models' Generation by Entropy-based Dynamic Temperature Sampling, Technical Report arXiv:2403.14541, arXiv (2024).
[12] Foodei, D. et al.: Semantic uncertainty in advanced decoding methods for LLM generation, Technical Report arXiv:2506.17296, arXiv (2025).
[13] Wang, X. et al.: Self-Consistency Improves Chain of Thought Reasoning in Language Models, *Proc. 11th International Conference on Learning Representations (ICLR)*, (online), available from <https://openreview.net/forum?id=1PL1NIMMrw> (2023).
[14] Chen, B. et al.: CodeT: Code Generation with Generated Tests, *Proc. 11th International Conference on Learning Representations (ICLR)*, (online), available from <https://openreview.net/forum?id=ktrw68Cmu9c> (2023).
[15] Le, H. et al.: CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning, *Advances in Neural Information Processing Systems 35 (NeurIPS)*, (online), available from <https://arxiv.org/abs/2207.01780> (2022).
[16] Madaan, A. et al.: Self-Refine: Iterative Refinement with Self-Feedback, *Advances in Neural Information Processing Systems 36 (NeurIPS)*, (online), available from <https://arxiv.org/abs/2303.17651> (2023).
[17] Zhu, Y. et al.: Uncertainty-Guided Chain-of-Thought for Code Generation with LLMs, Technical Report arXiv:2503.15341, arXiv (2025).
[18] Kadavath, S. et al.: Language Models (Mostly) Know What They Know, Technical Report arXiv:2207.05221, arXiv (2022).
[19] Xiong, M. et al.: Can LLMs Express Their Uncer-

tainty? An Empirical Evaluation of Confidence Elicitation in LLMs, *Proc. 12th International Conference on Learning Representations (ICLR)*, (online), available from <https://openreview.net/forum?id=gjeQKFxFpZ> (2024).

- [20] Gao, X. et al.: SPUQ: Perturbation-Based Uncertainty Quantification for Large Language Models, *Proc. 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 2336–2346 (online), DOI: 10.18653/v1/2024.eacl-long.143 (2024).
- [21] Kwon, W. et al.: Efficient Memory Management for Large Language Model Serving with PagedAttention, *Proc. 29th Symposium on Operating Systems Principles (SOSP)*, pp. 611–626 (online), available from <https://arxiv.org/abs/2309.06180> (2023).